

Olist E-Commerce Analytics

Revenue & Seller Performance — End-to-End Project Documentation

MySQL • Excel • Python • Groq LLM (AI) • Power BI

This document captures the full build process: every SQL query, Excel formula, and DAX measure written; every bug found and fixed; the business findings; and the reasoning behind each data-quality decision — intended as both a project record and interview prep reference.

Contents

- 1. Business Problem & Project Architecture
- 2. Key Findings
- 3. SQL Layer — Schema & Queries
- 4. Excel Layer — Formulas & Structure
- 5. Python Layer — EDA & Statistical Testing
- 6. AI Layer — LLM-Generated Executive Summary
- 7. Power BI — Data Model & DAX Measures
- 8. Power BI — Dashboard Screenshots
- 9. Bugs Found & Fixed (Full Log)
- 10. Data Quality Decisions & Assumptions
- 11. What I'd Improve With More Time
- 12. Interview Prep — Likely Questions & Answers

1. Business Problem & Project Architecture

Olist connects small sellers to major online marketplaces in Brazil. Leadership's underlying question: **is our revenue built on a stable foundation, or are we exposed to risk we don't have visibility into?** This splits into two concrete questions:

- Which sellers is the business financially dependent on, and are they reliable?
- Are customers being retained, or is growth entirely dependent on constant new acquisition?

Delivery reliability was investigated as a likely driver of both.

Architecture — each tool's role

Stage	Tool	Role
Data ingestion & modeling	MySQL	Loaded 9 raw CSVs (~1.5M rows) via Python/SQLAlchemy ETL, cleaned data types, added
Quick-look reporting	Excel	Pivot tables + formula-driven KPI summary sheet, independently replicating SQL segmentati
Deeper analysis	Python	EDA, delivery-delay distribution, statistical t-test linking lateness to review score
Insight generation	Groq LLM (Llama 3.3 70B)	Natural-language executive summary grounded strictly in validated findings, not raw data
Executive dashboard	Power BI	Star-schema model (fact/dim tables + date dimension), 3 interactive report pages

2. Key Findings

41% of total revenue is concentrated in "High Revenue – At Risk" sellers

Top-quartile revenue sellers with weak reviews or poor on-time delivery. The business is financially dependent on sellers who could churn or underperform with little warning.

39% of customers fall into "Lost" or "At Risk" RFM segments (vs. 21% "Champions")

9,125 Lost + 1,814 At Risk out of 27,741 total unique customers — a real retention problem, not just an acquisition one.

Late deliveries are associated with a 1.66-point drop in average review score

4.21/5 (on-time) vs 2.55/5 (late), t-statistic 90.50, $p < 0.0001$, $n = 110,013$. Statistically significant, not random noise.

Actual delivery runs ~12 days earlier than estimated, across every top product category

Olist's delivery estimates appear conservatively padded — may be setting unnecessarily low customer expectations.

7.9% of delivered orders arrive later than the estimated date

A relatively small share of orders, but one with an outsized effect on satisfaction (see review-score finding above).

One-sentence summary: "Identified that 41% of revenue is concentrated in seller relationships carrying real reliability risk, and 39% of customers are disengaging — then quantified delivery performance as a measurable driver of both, using SQL, Python statistical testing, and an interactive Power BI dashboard to make the findings actionable."

3. SQL Layer — Schema & Queries

3.1 Entity Relationships

customers —< orders —< order_items >— products / sellers | orders —< order_payments, order_reviews |
products —< category_translation

3.2 Type Cleanup (ALTER TABLE)

Applied to all 9 tables after Python-loaded CSVs defaulted every column to TEXT: converted date columns to DATETIME, IDs to VARCHAR(32), money fields to DECIMAL(10,2), added primary keys, then foreign keys last (once all primary keys existed). Example:

```
ALTER TABLE olist_orders_dataset
MODIFY COLUMN order_purchase_timestamp DATETIME,
MODIFY COLUMN order_id VARCHAR(32) NOT NULL,
MODIFY COLUMN customer_id VARCHAR(32) NOT NULL,
MODIFY COLUMN order_status VARCHAR(20) NOT NULL;

ALTER TABLE olist_orders_dataset ADD PRIMARY KEY (order_id);

ALTER TABLE olist_order_items_dataset
ADD CONSTRAINT fk_items_order FOREIGN KEY (order_id)
REFERENCES olist_orders_dataset(order_id);
```

Decision: used VARCHAR over ENUM for order_status — ENUM hardcodes business logic into the schema; a new status value would silently break inserts. VARCHAR is safer for anything not truly fixed forever.

3.3 Monthly Revenue Trend (window functions)

```
WITH revenue AS (
  SELECT DATE_FORMAT(od.order_purchase_timestamp, '%Y-%m') AS month,
         ROUND(SUM(oi.price), 2) AS revenue
  FROM olist_order_items_dataset oi
  JOIN olist_orders_dataset od ON oi.order_id = od.order_id
  WHERE od.order_status = 'delivered'
  GROUP BY DATE_FORMAT(od.order_purchase_timestamp, '%Y-%m')
)
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month) AS running_total,
       LAG(revenue) OVER (ORDER BY month) AS previous_revenue,
       ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
             / LAG(revenue) OVER (ORDER BY month) * 100, 2) AS mom_growth_pct
FROM revenue
ORDER BY month;
```

Bug caught: first draft calculated growth % backwards as (previous / current), which mislabels "previous month as % of current" as growth. Correct formula divides by the OLD value: (current - previous) / previous.

3.4 RFM Customer Segmentation

```
WITH customer_orders AS (
  SELECT c.customer_unique_id, o.order_id, o.order_purchase_timestamp, oi.price
  FROM olist_orders_dataset o
  JOIN olist_customers_dataset c ON c.customer_id = o.customer_id
  JOIN olist_order_items_dataset oi ON oi.order_id = o.order_id
  WHERE o.order_status = 'delivered'
),
rfm_base AS (
  SELECT customer_unique_id,
```

```

        DATEDIFF((SELECT MAX(order_purchase_timestamp) FROM olist_orders_dataset),
                 MAX(order_purchase_timestamp)) AS recency_days,
        COUNT(DISTINCT order_id) AS frequency,
        ROUND(SUM(price), 2) AS monetary
    FROM customer_orders
    GROUP BY customer_unique_id
),
rfm_scored AS (
    SELECT *,
           NTILE(5) OVER (ORDER BY recency_days ASC) AS r_score,
           NTILE(5) OVER (ORDER BY frequency DESC) AS f_score,
           NTILE(5) OVER (ORDER BY monetary DESC) AS m_score
    FROM rfm_base
)
SELECT *,
       CASE
           WHEN r_score <= 2 AND f_score <= 2 AND m_score <= 2 THEN 'Champions'
           WHEN r_score <= 2 AND f_score >= 3 THEN 'Loyal Customers'
           WHEN r_score >= 4 AND f_score <= 2 THEN 'At Risk'
           WHEN r_score >= 4 AND f_score >= 4 THEN 'Lost'
           ELSE 'Needs Attention'
       END AS customer_segment
FROM rfm_scored
ORDER BY monetary DESC;

```

Key trap avoided: grouped by customer_unique_id, NOT customer_id. In this dataset, customer_id is generated per-order, not per-person — grouping by it would make every repeat customer look like a one-time buyer.

3.5 Seller Performance Scorecard (final, corrected version)

```
WITH seller AS (
    SELECT oi.seller_id, oi.order_id, oi.price, oi.freight_value,
           od.order_status, od.order_delivered_customer_date, od.order_estimated_delivery_date
    FROM olist_order_items_dataset oi
    JOIN olist_orders_dataset od ON oi.order_id = od.order_id
    WHERE od.order_status = 'delivered' -- added after bug fix, see Section 9
),
seller_calculation AS (
    SELECT seller_id,
           COUNT(DISTINCT order_id) AS orders_count,
           COUNT(*) AS items_sold,
           ROUND(SUM(price), 2) AS gross_revenue,
           SUM(CASE WHEN order_status='delivered' THEN 1 ELSE 0 END) AS delivered_orders,
           SUM(CASE WHEN order_status='delivered'
                    AND order_delivered_customer_date <= order_estimated_delivery_date
                    THEN 1 ELSE 0 END) AS on_time_deliveries
    FROM seller
    GROUP BY seller_id
),
seller_reviews AS (
    SELECT oi.seller_id, ROUND(AVG(r.review_score), 2) AS avg_review_score
    FROM olist_order_items_dataset oi
    JOIN olist_order_reviews_dataset r ON oi.order_id = r.order_id
    GROUP BY oi.seller_id
),
scored AS (
    SELECT sc.*, sr.avg_review_score,
           RANK() OVER (ORDER BY gross_revenue DESC) AS revenue_rank,
           NTILE(4) OVER (ORDER BY gross_revenue DESC) AS revenue_quartile
    FROM seller_calculation sc
    LEFT JOIN seller_reviews sr ON sc.seller_id = sr.seller_id
)
SELECT *,
       CASE
           WHEN revenue_quartile = 1 AND avg_review_score >= 4.0
                AND on_time_deliveries/NULLIF(delivered_orders,0) >= 0.90 THEN 'Star Seller'
           WHEN revenue_quartile = 1 THEN 'High Revenue - At Risk'
           WHEN revenue_quartile IN (3,4) AND avg_review_score >= 4.5 THEN 'Hidden Gem'
           ELSE 'Standard'
       END AS seller_health_segment
FROM scored
ORDER BY gross_revenue DESC;
```

18% platform take rate applied elsewhere as a documented "profitability proxy" — Olist's public dataset has no real cost-of-goods or commission schedule, so true profit cannot be computed; this assumption is explicitly labeled, never presented as fact.

4. Excel Layer — Formulas & Structure

Workbook structure: Summary (KPIs + charts), Raw_Monthly_Revenue, Raw_Seller_Performance, Raw_RFM_Customers, Pivot_Seller_Segments, Pivot_RFM_Segments — raw data always kept separate from calculations.

4.1 Seller Performance — replicated SQL logic as Excel formulas

```
on_time_rate (H2)      = IFERROR(F2/E2, 0)
revenue_quartile (I2)  = ROUNDUP(RANK.EQ(D2,$D$2:$D$3096,0)/(COUNT($D$2:$D$3096)/4),0)
seller_health_segment (J2) =
  IF(AND(I2=1,G2>=4,H2>=0.9),"Star Seller",
    IF(AND(I2=1,OR(G2<4,H2<0.9)),"High Revenue - At Risk",
      IF(AND(OR(I2=3,I2=4),G2>=4.5),"Hidden Gem","Standard")))
```

4.2 Summary Sheet KPIs (final, label-based versions)

```
Total Revenue      = SUM(Raw_Monthly_Revenue!B2:B24)
Total Sellers       = COUNTA(Raw_Seller_Performance!A2:A3096)

% Revenue from At-Risk Sellers =
  SUMIF(Pivot_Seller_Segments!$A:$A,"High Revenue - At Risk",Pivot_Seller_Segments!$B:$B)
  / SUM(Raw_Seller_Performance!$D$2:$D$3096)

% Customers Lost/At Risk =
  (SUMIF(Pivot_RFM_Segments!$A:$A,"At Risk",Pivot_RFM_Segments!$B:$B)
  +SUMIF(Pivot_RFM_Segments!$A:$A,"Lost",Pivot_RFM_Segments!$B:$B))
  / COUNTA(Raw_RFM_Customers!$A$2:$A$27742)
```

Design decision: switched from pivot-row-position references (e.g. B5/B8) to SUMIF label lookups. Pivot row order shifts every refresh/resort — position-based formulas broke twice during this project (see Section 9) before switching to label-based lookups, which are immune to reordering.

Final validated Summary KPIs: Total Revenue 13,221,498.11 | Total Sellers 3,095 | % Revenue from At-Risk Sellers 41% | % Customers Lost/At Risk 39%

5. Python Layer — EDA & Statistical Testing

Scope deliberately limited to EDA, visualization, and statistical testing — no ML/predictive modeling, per project scoping decision.

5.1 MySQL connection & merged order-level table

```
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

url = URL.create(drivername="mysql+pymysql", username="root",
                 password="...", host="localhost", port=3306,
                 database="olist_ecommerce")
engine = create_engine(url)

df = orders.merge(order_items, on='order_id', how='inner')
df = df.merge(products, on='product_id', how='left')
df = df.merge(sellers, on='seller_id', how='left')
df = df.merge(customers, on='customer_id', how='left')
df = df[df['order_status'] == 'delivered'].copy() # consistency with SQL/Excel
```

URL.create() used instead of an f-string connection URL — a raw string broke when the MySQL password contained an '@' character, which the parser mistook for the host separator.

5.2 Delivery delay + late-delivery flag

```
df['is_late'] = (df['order_delivered_customer_date']
                > df['order_estimated_delivery_date']).astype(int)
df['delivery_delay_days'] = (df['order_delivered_customer_date']
                             - df['order_estimated_delivery_date']).dt.days

# Result: 92.1% on-time, 7.9% late (n=110,197) – realistic imbalanced split
```

5.3 Statistical test — delivery lateness vs review score

```
from scipy import stats

merged = df.merge(reviews, on='order_id')
on_time_scores = merged[merged['is_late']==0]['review_score']
late_scores = merged[merged['is_late']==1]['review_score']

t_stat, p_value = stats.ttest_ind(on_time_scores, late_scores, equal_var=False)
# On-time avg: 4.211 | Late avg: 2.549
# T-statistic: 90.50 | P-value: ~0.0000000000 (< 0.0001)
# Sample sizes: on-time 101,453 | late 8,560
```

Caveat worth stating precisely: a t-test proves the difference is real and not due to chance — it does not prove lateness directly causes low ratings (a shared cause like a damaged product could drive both).

5.4 Category revenue with average delay

```
cat_revenue = df.groupby('product_category_name').agg(
    total_revenue=('price', 'sum'),
    order_count=('order_id', 'nunique'),
    avg_delay=('delivery_delay_days', 'mean')
).sort_values('total_revenue', ascending=False).head(10)
```

Insight: avg_delay landed around -11 to -12 days for every top category, consistently — confirming delivery estimates are systematically padded, not just occasionally early.

6. AI Layer — LLM-Generated Executive Summary

Used Groq (Llama 3.3 70B) to turn already-validated findings into a written executive summary — the LLM was never given raw data to analyze itself, only proven conclusions, to minimize hallucination risk.

```
from groq import Groq
import os
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

prompt = f"""You are a senior business analyst presenting findings to Olist's
leadership team. Based STRICTLY on the data below, write a concise executive
summary (300-400 words) with:
1. A 2-3 sentence overview of business health
2. Three key risks, ranked by urgency
3. Three specific, actionable recommendations tied directly to the data
Do not invent any numbers not present in the data below.

DATA:
{business_findings}
"""

response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.3,
)
```

Design choices worth explaining: temperature kept low (0.3) since this is a factual report, not creative writing; the prompt explicitly instructs against inventing numbers — a real prompt-engineering technique to reduce hallucination.

Generated Executive Summary (actual output)

Olist's business health is characterized by stabilized monthly growth and a sizable revenue base of R\$13,221,498.11, with 3,095 active sellers contributing to this figure. However, this growth is accompanied by notable risks that require attention. The company's revenue trajectory is marked by fluctuations, ranging from -13% to +52% month-over-month, indicating a need for strategies to mitigate volatility.

Three key risks, ranked by urgency:

1. Concentration of revenue in "High Revenue - At Risk" sellers: 41% of total revenue is generated by sellers with weak reviews or poor on-time delivery, posing a significant threat to long-term sustainability.
2. Customer churn: 39% of customers are categorized as "At Risk" or "Lost", highlighting the need for targeted retention strategies.
3. Delivery satisfaction: 7.9% of delivered orders arrive later than estimated, resulting in significantly lower customer satisfaction ratings (2.55/5 vs 4.21/5 for on-time orders).

Recommendations:

1. Develop a seller performance improvement program targeting "High Revenue - At Risk" sellers.
2. Implement targeted customer retention initiatives leveraging RFM insights.
3. Refine delivery estimates and communication, which currently run ~12 days earlier than promised.

7. Power BI — Data Model & DAX Measures

Star schema: 3 fact tables (Fact_OrderItems, Fact_Payments, Fact_Reviews) and 4 dimension tables (Dim_Orders, Dim_Products, Dim_Sellers, Dim_Date), plus Dim_CustomerRFM imported directly from the validated SQL export rather than rebuilt in DAX.

7.1 Core measures

```
Total Revenue      = SUM(Fact_OrderItems[price])
Total Orders       = DISTINCTCOUNT(Fact_OrderItems[order_id])
Avg Order Value    = DIVIDE([Total Revenue], [Total Orders])
Total Sellers      = DISTINCTCOUNT(Dim_Sellers[seller_id])
Total Customers    = DISTINCTCOUNT(Dim_CustomerRFM[customer_unique_id])
Avg Review Score   = AVERAGE(Fact_Reviews[review_score])
```

DIVIDE() used instead of "/" throughout — returns blank instead of erroring on a zero denominator, same class of problem as the #DIV/0! bug hit in Excel, handled properly from the start in DAX.

7.2 Delivery measures

```
Delivered Orders =
CALCULATE(DISTINCTCOUNT(Dim_Orders[order_id]), Dim_Orders[order_status]="delivered")

On-Time Deliveries =
CALCULATE(DISTINCTCOUNT(Dim_Orders[order_id]),
    Dim_Orders[order_status]="delivered",
    Dim_Orders[order_delivered_customer_date] <= Dim_Orders[order_estimated_delivery_date])

On-Time Rate = DIVIDE([On-Time Deliveries], [Delivered Orders])
```

7.3 Seller calculated columns (on Dim_Sellers)

```
Seller Revenue = CALCULATE(SUM(Fact_OrderItems[price]), RELATEDTABLE(Fact_OrderItems))

Revenue Quartile =
VAR SellerRank = RANKX(ALL(Dim_Sellers), Dim_Sellers[Revenue], , DESC)
VAR TotalSellers = COUNTROWS(ALL(Dim_Sellers))
RETURN ROUNDUP(SellerRank / (TotalSellers / 4), 0)

Seller On-Time Rate =
VAR DeliveredForSeller =
    CALCULATE(DISTINCTCOUNT(Dim_Orders[order_id]), RELATEDTABLE(Fact_OrderItems),
        Dim_Orders[order_status]="delivered")
VAR OnTimeForSeller =
    CALCULATE(DISTINCTCOUNT(Dim_Orders[order_id]), RELATEDTABLE(Fact_OrderItems),
        Dim_Orders[order_status]="delivered",
        Dim_Orders[order_delivered_customer_date] <= Dim_Orders[order_estimated_delivery_date])
RETURN DIVIDE(OnTimeForSeller, DeliveredForSeller)

Seller Avg Review = CALCULATE(AVERAGE(Fact_Reviews[review_score]), RELATEDTABLE(Fact_OrderItems))

Seller Health Segment =
IF(Dim_Sellers[Revenue Quartile]=1 && Dim_Sellers[Seller Avg Review]>=4
    && Dim_Sellers[Seller On-Time Rate]>=0.9, "Star Seller",
    IF(Dim_Sellers[Revenue Quartile]=1, "High Revenue - At Risk",
        IF(Dim_Sellers[Revenue Quartile]>=3 && Dim_Sellers[Seller Avg Review]>=4.5,
            "Hidden Gem", "Standard"))))
```

7.4 Risk / KPI measures

```
Revenue At-Risk Sellers =  
CALCULATE([Total Revenue], Dim_Sellers[Seller Health Segment]="High Revenue - At Risk")  
Pct Revenue At Risk = DIVIDE([Revenue At-Risk Sellers], [Total Revenue])  
  
Customers Lost Or At Risk =  
CALCULATE(DISTINCTCOUNT(Dim_CustomerRFM[customer_unique_id]),  
    Dim_CustomerRFM[customer_segment] IN {"Lost","At Risk"})  
Pct Customers At Risk =  
DIVIDE([Customers Lost Or At Risk], DISTINCTCOUNT(Dim_CustomerRFM[customer_unique_id]))
```

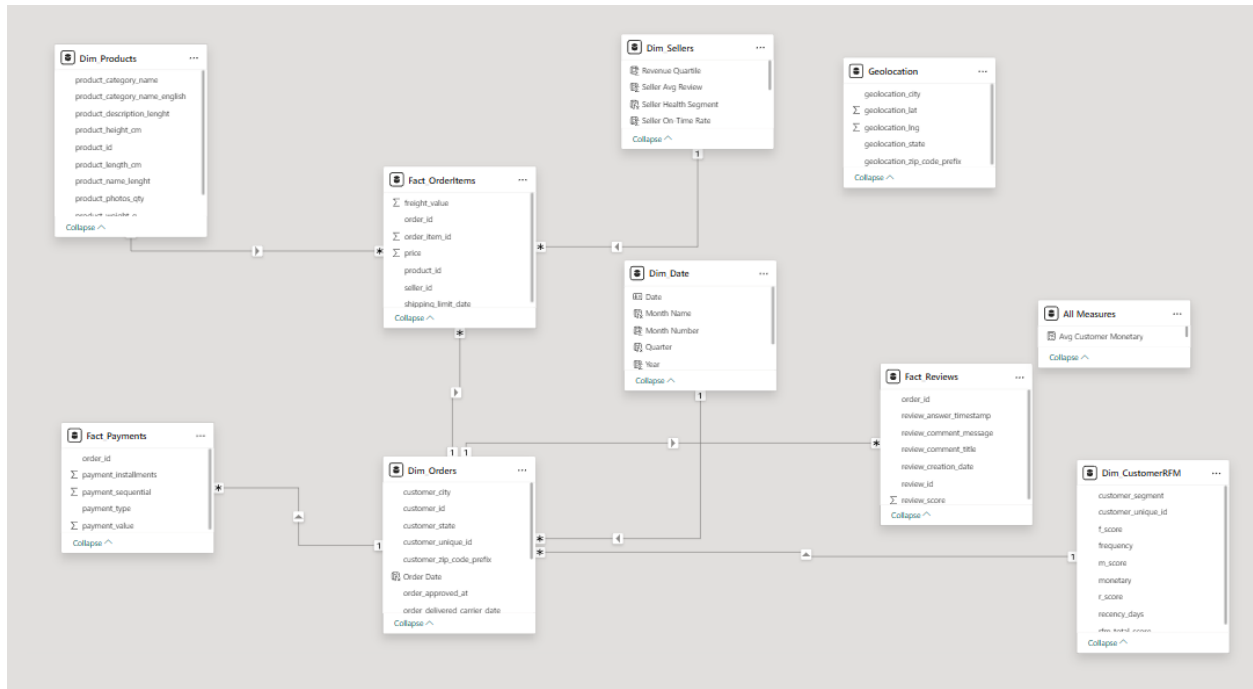
7.5 Date dimension

```
Dim_Date = CALENDAR(DATE(2016,1,1), DATE(2018,12,31))  
Year = YEAR(Dim_Date[Date])  
Month Number = MONTH(Dim_Date[Date])  
Month Name = FORMAT(Dim_Date[Date], "MMM") -- sorted by Month Number  
Year-Month = FORMAT(Dim_Date[Date], "YYYY-MM") -- sorted by YearMonthSort  
YearMonthSort = YEAR(Dim_Date[Date]) * 100 + MONTH(Dim_Date[Date])  
  
-- On Dim_Orders, needed to relate cleanly to Dim_Date:  
Order Date = DATE(YEAR([order_purchase_timestamp]), MONTH([order_purchase_timestamp]),  
    DAY([order_purchase_timestamp]))
```

Table marked as an official Date Table (Table Tools → Mark as Date Table) to enable native time-intelligence DAX functions.

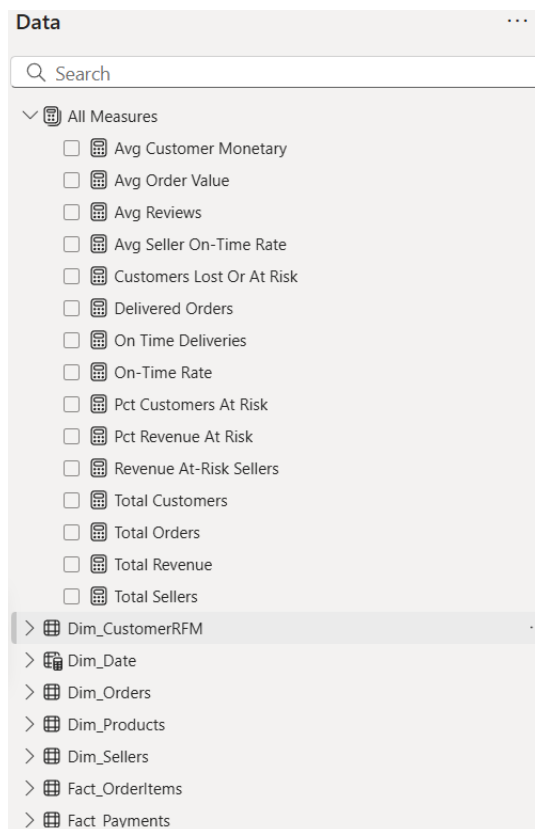
8. Power BI — Dashboard Screenshots

8.1 Data Model (Star Schema)



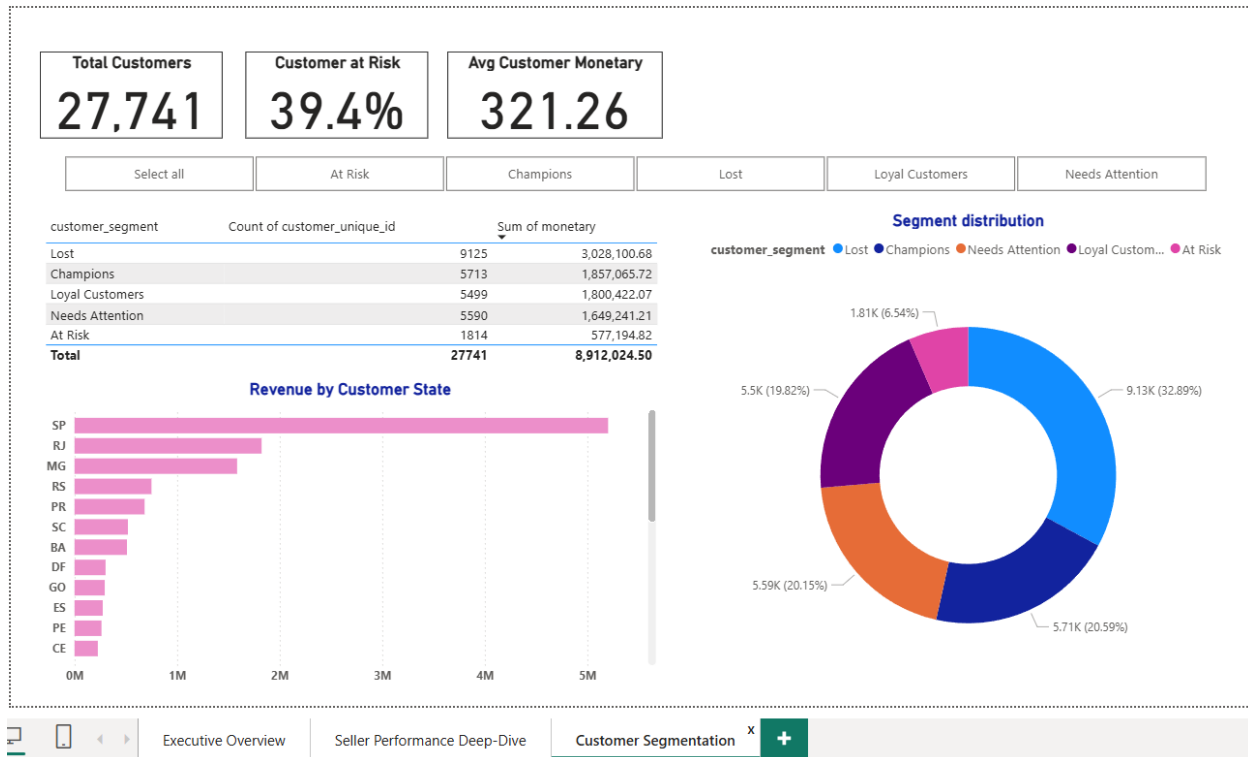
3 fact tables + 4 dimension tables (Dim_CustomerRFM imported directly from validated SQL export). Geolocation intentionally left disconnected to avoid fan-out duplication.

8.2 Executive Overview



4 cross-validated KPIs, revenue trend (2017+ only, pilot months excluded), seller health segment donut. Cross-check: KPI 41.0% = donut 41.02% — same figure computed two independent ways.

8.3 Seller Performance Deep-Dive



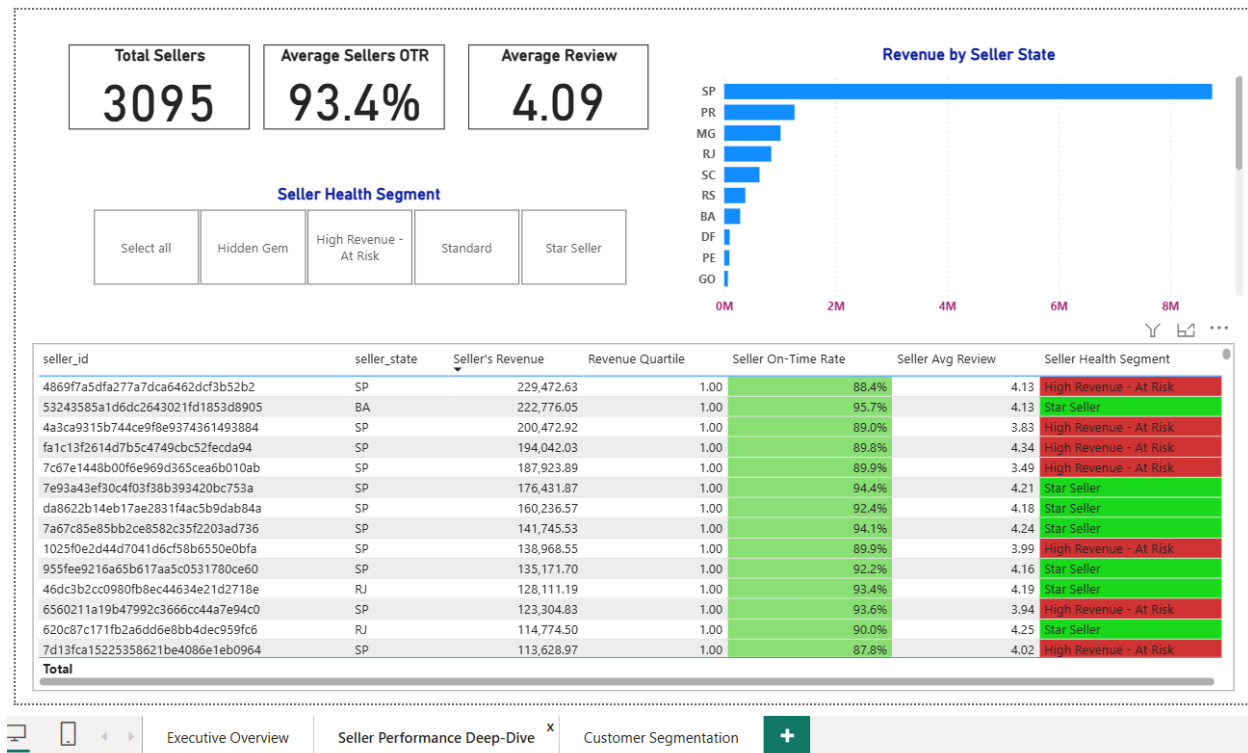
Executive Overview

Seller Performance Deep-Dive

Customer Segmentation x

Interactive seller table with conditional-formatted health segment and on-time rate, tile slicer, revenue-by-state breakdown.

8.4 Customer Segmentation (RFM)



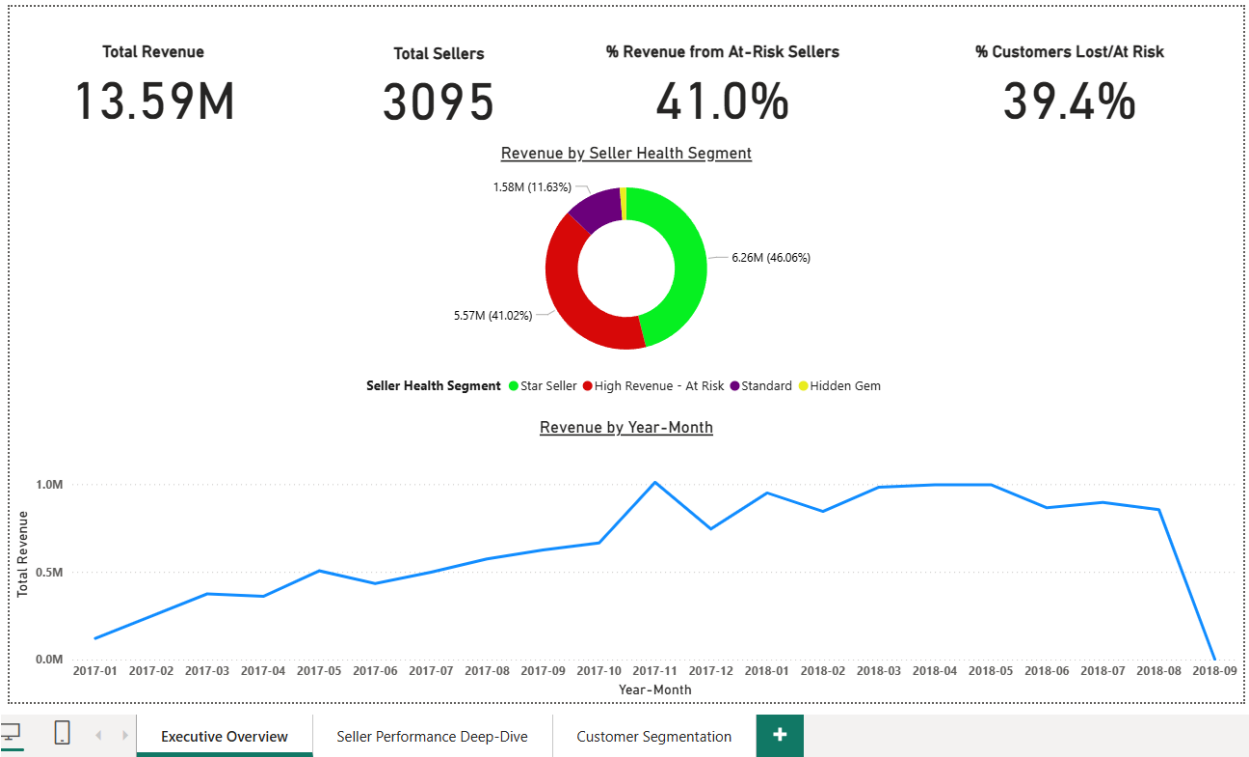
Executive Overview

Seller Performance Deep-Dive x

Customer Segmentation

RFM segment table and donut matching validated Excel pivot exactly (Lost 9,125 / Champions 5,713 / Loyal 5,499 / Needs Attention 5,590 / At Risk 1,814 = 27,741 total).

8.5 Available Measures



Full measures list under the _Measures home table.

9. Bugs Found & Fixed — Full Log

Documenting these deliberately — catching and explaining real bugs is a stronger signal of analytical rigor than a project with no visible debugging at all.

1. MySQL import wizard silently under-loaded data

Problem: Workbench's Table Data Import Wizard loaded only a small fraction of rows per table (e.g. 258 of 99,441 customers) with no visible error, just a batching failure buried in the action log.

Fix: Abandoned the GUI wizard; switched to a Python (pandas + SQLAlchemy) ETL loader script — more reliable, reusable, and closer to real-world ETL practice.

2. MySQL connection string broke on special-character password

Problem: A plain f-string connection URL silently mis-parsed a password containing '@', which was interpreted as part of the hostname, producing a nonsensical 'Can't connect to ...@localhost' error.

Fix: Switched to SQLAlchemy's URL.create() which properly encodes special characters.

3. Revenue definition mismatch: 13,221,498.11 (SQL/Excel monthly) vs 13,591,643.70 (SQL seller performance)

Problem: The seller-performance query's base CTE never filtered WHERE order_status = 'delivered', while the monthly revenue query did — same underlying data, two different definitions of 'revenue' used unknowingly across the project.

Fix: Standardized 'delivered orders only' as the single revenue definition, enforced consistently across SQL, Excel, Python, and Power BI. Documented explicitly in the README.

4. Excel #DIV/0! in 125 sellers' on_time_rate

Problem: on_time_rate = F2/E2 divided by zero for sellers with zero delivered orders (all orders cancelled), breaking their segment classification too.

Fix: Changed formula to IFERROR(F2/E2, 0) — zero successful deliveries treated as a 0% on-time rate, which is also the correct business interpretation.

5. Excel Summary KPI formulas referenced wrong pivot rows

Problem: Position-based references (e.g. B4, B5) silently pointed at the wrong segment after the pivot was refreshed and row order shifted — produced a plausible-looking but wrong number with no error thrown.

Fix: Rebuilt as SUMIF label lookups instead of hardcoded cell positions — immune to row reordering.

6. Pivot table showed stale results after source data was fixed

Problem: Excel pivot tables cache their results; editing the underlying formula (the #DIV/0! fix) didn't auto-propagate. Produced a nonsensical 19,360% KPI because the formula divided against the still-stale cached Grand Total.

Fix: Right-click pivot → Refresh (or Data → Refresh All) required after any source-data change.

7. One blank row in RFM customer segment pivot

Problem: A stray row with only customer_unique_id populated (a CSV export artifact) surfaced as an unlabeled "(blank)" segment.

Fix: Identified and deleted the stray row; pivot total correctly dropped from 27,742 to 27,741.

8. Python: ModuleNotFoundError despite packages being 'installed'

Problem: Multiple separate Python installs on the same machine (system Python, uv-managed .venv, and later a hidden Anaconda-style global Python at C:\Program Files\Python311) caused the notebook kernel to silently run a different interpreter than the terminal, even when paths looked correct.

Fix: Diagnosed by comparing sys.executable from inside the notebook against the terminal's active Python directly, instead of trusting the kernel-selector UI label.

9. Power BI revenue trend chart showed a single data point

Problem: Dim_Date was built as pure DATE values (CALENDAR()), but Dim_Orders[order_purchase_timestamp] is a full DATETIME with time-of-day — almost no rows matched exactly, so nearly every order fell into an unrelated '(Blank)' bucket.

Fix: Added a date-only 'Order Date' column on Dim_Orders (stripping time), relaced the relationship to join on that instead of the raw timestamp.

10. Power BI trend chart x-axis sorted by value, not chronologically

Problem: Year-Month is a text field; without a numeric sort key Power BI defaulted to sorting by the chart's own measure value, scrambling the month order.

Fix: Added a YearMonthSort numeric column, applied via Sort by Column on Year-Month — fixes every visual using that field, not just one chart.

11. Power BI KPI card showed count of 1 instead of 3,095

Problem: A table row was left selected (cross-filter highlight), which filtered the Total Sellers card down to just that one row — not a build error, an interaction state left on.

Fix: Deselected the row; optionally disabled cross-filtering from that table to that card via Edit Interactions.

12. Power BI Total Customers KPI (96K) disagreed with the RFM table total (27,741)

Problem: The measure counted DISTINCTCOUNT(Dim_Orders[customer_unique_id]) — all orders regardless of status — while the RFM table was scoped to delivered-only, same class of bug as the SQL revenue mismatch, recurring in a third tool.

Fix: Repointed the measure to DISTINCTCOUNT(Dim_CustomerRFM[customer_unique_id]), which is already correctly scoped.

13. Stray '(Blank)' tile in the customer segment slicer

Problem: Customers who exist in Dim_Orders (via non-delivered orders) but have no matching row in Dim_CustomerRFM (delivered-only scope) generate an automatic blank placeholder member on the relationship.

Fix: Added a visual-level filter excluding blank customer_segment values from the slicer (and other visuals using that field).

10. Data Quality Decisions & Assumptions

Revenue standardized to "delivered orders only", everywhere

Cancelled/unfulfilled order value excluded consistently across SQL, Excel, Python, and Power BI, after the mismatch bug in Section 9 exposed the risk of not doing so.

Pilot-phase data excluded from trend analysis (Sep 2016 – Jan 2017)

Fewer than 10 orders total across these months; including them produces misleading growth percentages (one month showed 1,000,000%+ month-over-month growth).

Final partial month excluded (Sep 2018)

Olist's real data collection tapers off mid-month, creating an artificial cliff in any revenue trend chart that includes it.

"Profitability" is a documented proxy (assumed 16-18% take rate), not a real figure

The public dataset has no seller cost-of-goods or Olist's actual commission schedule — an assumption is used and explicitly labeled as such, never presented as fact.

customer_unique_id used for all customer-level analysis, never customer_id

customer_id is generated per order in this dataset, not per person; using it directly would make every repeat customer look like a one-time buyer, corrupting RFM and retention analysis.

Review scores counted for all reviews regardless of order outcome, not just delivered orders

Deliberate: review score reflects overall seller reputation, not only fulfilled-order experience. Stated as a conscious choice, not an oversight.

11. What I'd Improve With More Time

- Formal cohort retention analysis — do customers who bought in month N return in N+1, N+2...
- Sentiment analysis (NLP) on review comment text to complement the star-rating findings
- Seller-level revenue forecasting / trend projection
- Predictive model for late-delivery risk at time of order placement (deliberately scoped out of this version)

12. Interview Prep — Likely Questions & Answers

Q: What problem does this project solve?

A: Two linked business questions: which sellers is the platform financially dependent on and are they reliable, and are customers being retained. Found 41% of revenue sits with 'at-risk' sellers and 39% of customers are Lost/At Risk, then statistically proved delivery reliability drives both.

Q: Why customer_unique_id instead of customer_id?

A: customer_id is generated per order in this dataset, not per person. Grouping by it would make every repeat customer look like a first-time buyer, breaking RFM and retention analysis entirely.

Q: Walk me through a bug you found and fixed.

A: Seller-level revenue (13.59M) disagreed with order-level revenue (13.22M) by ~370K. Traced it to one query missing the delivered-only filter that every other query already had. Fixed it and standardized the definition across all five tools — caught by deliberately cross-checking totals, not by accident.

Q: What does the t-test result actually tell you?

A: That the review-score gap between on-time and late deliveries (4.21 vs 2.55, $p < 0.0001$) is real and very unlikely due to random chance, given the large sample sizes. It does NOT prove lateness directly causes the low rating — correlation, not proven causation; a shared cause (e.g. a damaged product) could drive both.

Q: Why not build a machine-learning model?

A: Deliberately scoped out to keep the project focused on defensible, explainable statistical and business analysis across five tools rather than spreading thin. Noted as a natural next step (late-delivery risk prediction) in 'What I'd improve.'

Q: How did you validate the Power BI numbers against your earlier work?

A: Built a cross-tool consistency table (Total Revenue, Total Sellers, % at-risk revenue, RFM totals) and checked SQL, Excel, and Power BI all agree — catching, for example, a Power BI Total Customers measure that was scoped differently from the RFM table right next to it on the same page.

Q: What would you do differently with more time?

A: Formal cohort retention analysis, NLP sentiment on review text, and a late-delivery risk prediction model — all deliberately scoped out to ship a focused, defensible v1 first.

This document is intended as a personal reference — read through Sections 9, 10, and 12 until the reasoning feels natural to explain out loud without re-reading it.